Forecasting unemployment rate, with GDP and ARMAX

2026 March 11

Function for ARMAX holdout set RMSE for 1-step ahead forecasts

```
#' @description
#' Moving 1-step ahead forecasts for holdout set, ARMAX
#' @param tsdata vector with training and holdout data
#' @param ntrain length of training set
#' @param order ARIMA order using for training set
#' @param method as used for arima applied to training set
#' @param traincoef the $coef component for arima applied to training set
#' @param include.mean flag as used for arima applied to training set
#' @param xreg matrix with predictors for arima()
#' @return holdout set rmse and 1-step forecasts
armax_fc = function(tsdata,ntrain,order,method,traincoef,include.mean,xreg,
   iprint=F)
{ obj = arima(tsdata,order=order,init=traincoef,fixed=traincoef,
  # next line for debugging, after debugging, replace with above line
  # obj <<- arima(tsdata,order=order,init=traincoef,fixed=traincoef,
     method=method, include.mean=include.mean, xreg=xreg,
     optim.control=list(maxit=0))
  # yt = y_{t|t-1} + innov ; fc = yt-innov
  fc = tsdata - obj$residuals
  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]
  if(iprint) print(cbind(holdout,holdout_fc))
  rmse = sqrt(mean((holdout-holdout_fc)^2))
  list(rmse=rmse, fc=holdout_fc)
}
```

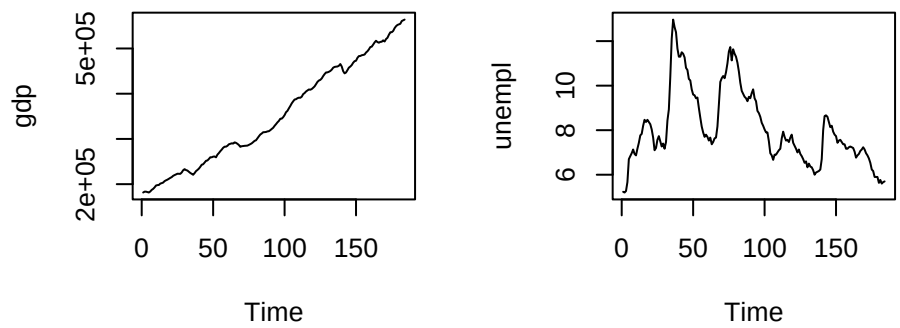
Step 1: Input data set and select training and holdout periods

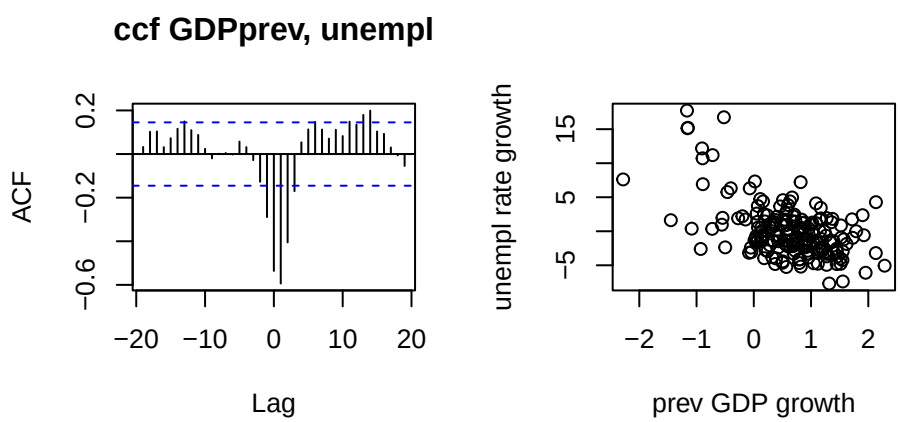
```
# Years 1961 to 2019 or 59 years and 236 quarters
df = read.csv("gdp-unemploy-FRED.csv", header=T, nrow=236)
names(df)
#> [1] "date"        "gdp"         "unempl"      "diflngdp"    "diflnunempl"

# Differences of log of consecutive values have been multiplied by 100

# A subset time series is used to show ARMAX with AR(1) residuals after regression.
# The period is different from in unemploy-gdp.Rmd
ibegin = 53;  iend = 236  # 1974 to 2019
print(df[ibegin,])
#>          date    gdp   unempl  diflngdp diflnunempl
```

```r
#> 53 1974-01-01 181552 5.233333 0.8241989   -5.574199

print(df[iend,])
#>           date      gdp unempl  diflngdp diflnunempl
#> 236 2019-10-01 563838.5    5.7 0.2850975   0.5865061

df = df[ibegin:iend,]
print(dim(df))
#> [1] 184   5

ntotal = nrow(df)
print(ntotal)
#> [1] 184

# First column is date
gdp = df[,2]; unempl = df[,3]

ntrain = floor(ntotal*0.7)
print(ntrain)
#> [1] 128

train = unempl[1:ntrain]
holdout = unempl[(ntrain+1):ntotal]
```

Step 2. Prediction with previous difference log(gdp) and difference log(unemploy)

```r
# Dataframe with previous differences, number of rows is ntrain-2
df2 = data.frame(
   gprev=df$diflngdp[2:(ntotal-1)],
   uprev=df$diflnunempl[2:(ntotal-1)], unempl=df$diflnunempl[3:ntotal])
print(dim(df2))
#> [1] 182   3

par(mfrow=c(2,2))
plot.ts(gdp)
plot.ts(unempl)
ccf(df2$gprev,df2$unempl,main="ccf GDPprev, unempl")
plot(df2$gprev,df2$unempl,xlab="prev GDP growth",ylab="unempl rate growth")
```





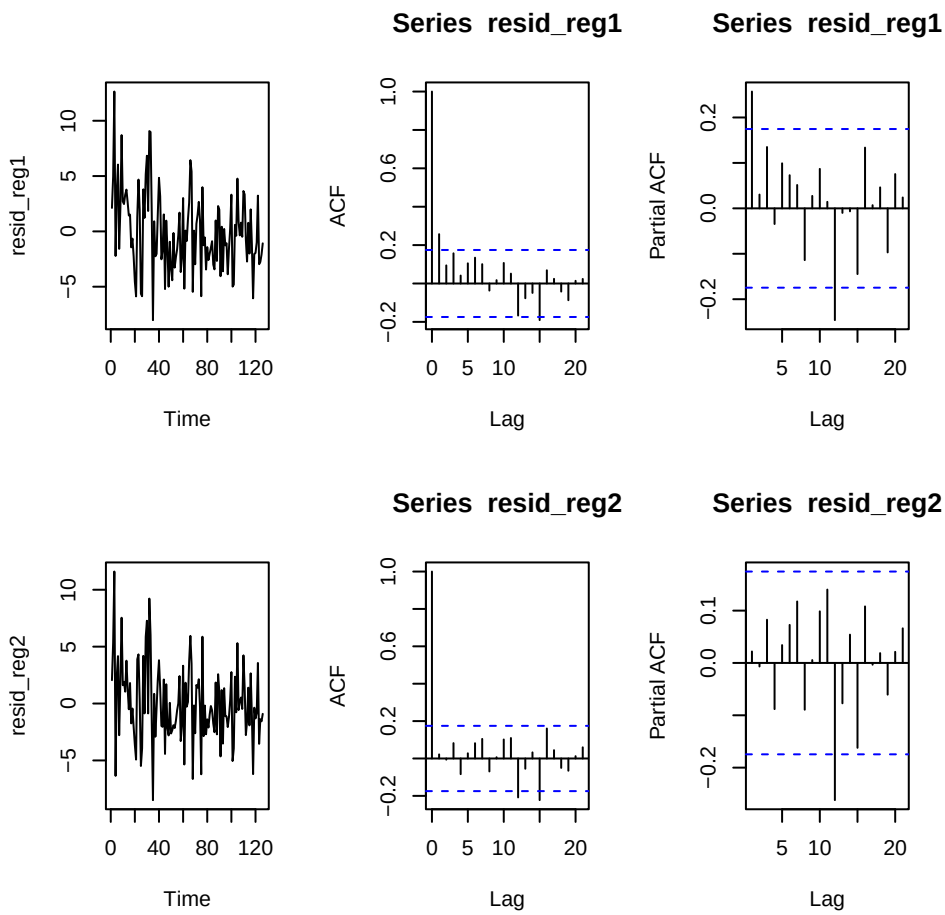

```
# Regression 1. Fit regression on previous diflngdp with training set
reg1 = lm(unempl ~ gprev, data=df2[1:(ntrain-2),])
print(summary(reg1))
#>
#> Call:
#> lm(formula = unempl ~ gprev, data = df2[1:(ntrain - 2), ])
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -8.0193 -2.3212 -0.3774  2.0891 12.6370
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   2.4342     0.4338   5.612 1.24e-07 ***
#> gprev        -3.2153     0.4213  -7.633 5.30e-12 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 3.568 on 124 degrees of freedom
#> Multiple R-squared:  0.3196, Adjusted R-squared:  0.3142
#> F-statistic: 58.26 on 1 and 124 DF,  p-value: 5.301e-12

# Predicted value for Y_t is  100 * ( log(unempl[t]) - log(unempl[t-1]) )
```

```r
# => predicted value for unempl[t] is exp(pred1/100) * unempl[t-1]
pred_reg1 = predict(reg1, newdata=df2[(ntrain-1):(ntotal-2),])
# 1-step ahead point forecast of unemployment rates for holdout set
unempl_reg1_fc = exp(pred_reg1/100) * unempl[ntrain:(ntotal-1)]

rmse_reg1 = sqrt(mean((holdout-unempl_reg1_fc)^2))
cat("rmse_reg1=", rmse_reg1,"\n")
#> rmse_reg1= 0.2304547


# Regression 2. Fit regression on previous diflngdp and diflnunempl with training set
reg2 = lm(unempl ~ gprev+uprev, data=df2[1:(ntrain-2),])
print(summary(reg2))
#>
#> Call:
#> lm(formula = unempl ~ gprev + uprev, data = df2[1:(ntrain - 2),
#>     ])
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -8.4887 -2.0678 -0.4198  2.0165 11.5975
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   1.6130     0.4632   3.482 0.000690 ***
#> gprev        -2.1361     0.4878  -4.379 2.52e-05 ***
#> uprev         0.3312     0.0859   3.856 0.000185 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 3.384 on 123 degrees of freedom
#> Multiple R-squared:  0.393,  Adjusted R-squared:  0.3831
#> F-statistic: 39.82 on 2 and 123 DF,  p-value: 4.632e-14

pred_reg2 = predict(reg2, newdata=df2[(ntrain-1):(ntotal-2),])
# 1-step ahead point forecasts of unemployment rate for holdout set
unempl_reg2_fc = exp(pred_reg2/100) * unempl[ntrain:(ntotal-1)]

rmse_reg2 = sqrt(mean((holdout-unempl_reg2_fc)^2))
cat("rmse_reg2=", rmse_reg2,"\n")
#> rmse_reg2= 0.2245129
```

Step 3. ARMAX with AR(1) for residuals

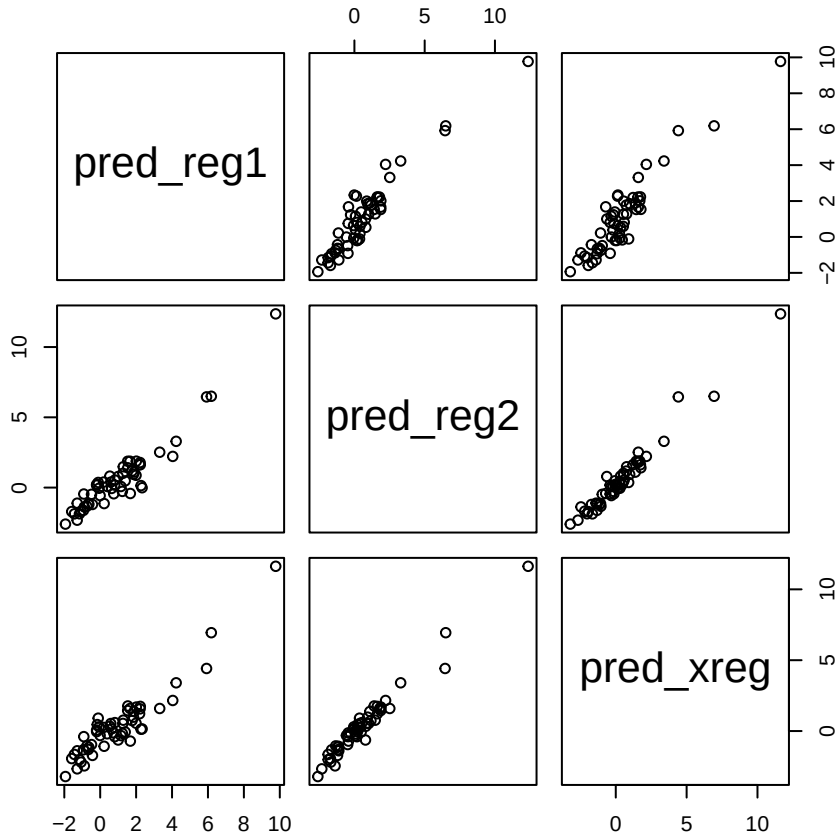
```r
resid_reg1 = reg1$residuals
resid_reg2 = reg2$residuals
par(mfrow=c(2,3))
plot.ts(resid_reg1)
acf(resid_reg1)
pacf(resid_reg1)
plot.ts(resid_reg2)
acf(resid_reg2)
```

```r
pacf(resid_reg2)
```

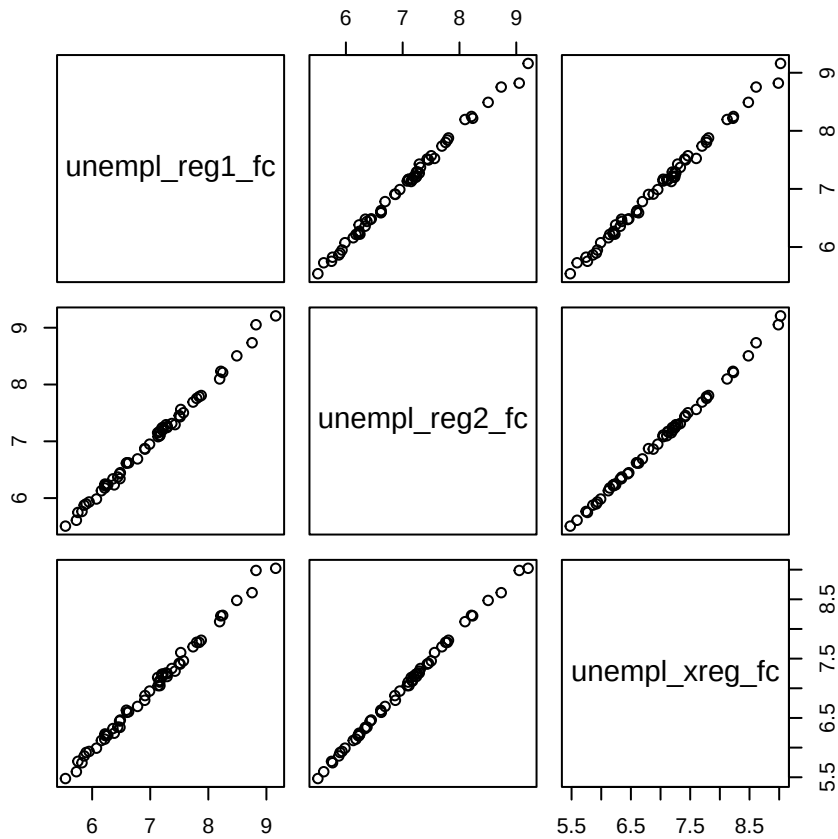



```r
library(forecast)
resid1_auto = auto.arima(resid_reg1,stationary=T,seasonal=F)
print(resid1_auto)
#> Series: resid_reg1
#> ARIMA(1,0,0) with zero mean
#>
#> Coefficients:
#>          ar1
#>       0.2559
#> s.e.  0.0858
#>
#> sigma^2 = 11.79:  log likelihood = -333.76
#> AIC=671.52   AICc=671.61   BIC=677.19

resid2_auto = auto.arima(resid_reg2,stationary=T,seasonal=F)
print(resid2_auto)
#> Series: resid_reg2
#> ARIMA(0,0,0) with zero mean
#>
#> sigma^2 = 11.18:  log likelihood = -330.86
```

```r
#> AIC=663.71   AICc=663.75   BIC=666.55

# For regression on previous GDP growth and previous unemployment rate growth,
# residual time series is close to white noise.
# For regression on previous GDP growth only, residual time series has
# weak serial dependence.

# ARX using reg1 with previous GDP growth as explanatory variable
train_arx = arima(df2$unempl[1:(ntrain-2)], order=c(1,0,0), method="CSS",
  include.mean=T, xreg=df2$gprev[1:(ntrain-2)])
print(train_arx)
#>
#> Call:
#> arima(x = df2$unempl[1:(ntrain - 2)], order = c(1, 0, 0), xreg = df2$gprev[1:(ntrain -
#>     2)], include.mean = T, method = "CSS")
#>
#> Coefficients:
#>          ar1  intercept  df2$gprev[1:(ntrain - 2)]
#>       0.3626     1.7012                    -2.2117
#> s.e.  0.1014     0.6119                     0.5500
#>
#> sigma^2 estimated as 11.45:  part log likelihood = -332.36

train_arx_ml = arima(df2$unempl[1:(ntrain-2)], order=c(1,0,0), method="ML",
  include.mean=T, xreg=df2$gprev[1:(ntrain-2)])
print(train_arx_ml)
#>
#> Call:
#> arima(x = df2$unempl[1:(ntrain - 2)], order = c(1, 0, 0), xreg = df2$gprev[1:(ntrain -
#>     2)], include.mean = T, method = "ML")
#>
#> Coefficients:
#>          ar1  intercept  df2$gprev[1:(ntrain - 2)]
#>       0.3608     1.7297                    -2.2084
#> s.e.  0.1007     0.6067                     0.5484
#>
#> sigma^2 estimated as 11.38:  log likelihood = -332.05,  aic = 672.1

# For df2, training set size decreased by 2 because of differences and lags
# Input parameter vector for arima applied to training set.
xreg_obj = armax_fc(df2$unempl, ntrain=(ntrain-2), order=c(1,0,0), method="CSS",
  traincoef=train_arx$coef, include.mean=T, xreg=df2$gprev)

pred_xreg = xreg_obj$fc
pairs(cbind(pred_reg1,pred_reg2,pred_xreg))
```




```
unempl_xreg_fc = exp(pred_xreg/100) * unempl[ntrain:(ntotal-1)]
pairs(cbind(unempl_reg1_fc,unempl_reg2_fc,unempl_xreg_fc))
```




```
rmse_xreg = sqrt(mean((holdout-unempl_xreg_fc)^2))

rmse_vec = c(rmse_reg1,rmse_reg2,rmse_xreg)
print(round(rmse_vec,3))
#> [1] 0.230 0.225 0.212

# Compare 1-step and 2-step ARMAX parameter estimation
bvec = reg1$coef
phi = resid1_auto$coef
cat("2-step estimation:", phi,bvec,"\n")
#> 2-step estimation: 0.2559129 2.434196 -3.215305

cat("1-step estimation:", train_arx$coef,"\n")
#> 1-step estimation: 0.3626291 1.701153 -2.211702

# Two-step estimation led to worse holdout set RMSE,
# likely because of small sample size without using information on
# dependence of parameter estimates.
```